

2026

HVAfvalruim bot



Chris Kuijper
Hogeschool van Amsterdam

Inhoud

Introductie.....	2
Belangrijke onderdelen	3
Origin One.....	3
Xarm6	3
Jetson Orin Nano super	4
Verloop project	5
Jetson starten.....	5
Obstakel 1	5
Obstakel 2	5
Oplossing	5
Codering	6
Code 1: beeld herkenning.....	6
Code 2: pin signal	10
Tekeningen.....	14
3d tekeningen	14
Elektrotechnische tekeningen	16
Vorbereidingen.....	27
Overige onderwerpen	28
Onderhoud.....	28
Obstakels	31
Te weinig kennis.....	31
ROS2.....	31
Conclusie	32

Introductie

Beste lezer de afgelopen weken heb ik met veel plezier mogen werken aan het HVAfval bot project van het HVA. Dit project maakt gebruik van beide de Origin One van Avular en de Xarm6 van Ufactory. Hierbij zal de Origin One werken als een beweegbaar plateau voor de Xarm6 deze zal verschillende soorten afval oppakken en in de een prullenbak stoppen om gebouwen schoner te maken. In dit document zal ik een uitleg geven van de code, de 3D-tekeningen en de elektrische tekeningen die nodig waren bij dit project. Verder zal ik ook de obstakels die ik tegenkwam tijdens dit project bespreken.

Belangrijke onderdelen

In dit stuk zal ik de 3 belangrijkste onderdelen bespreken die nodig zijn voor dit project.

Origin One

Zoals eerder opgenoemd is de Origin One het bewegende plateau voor dit project. De Origin is een robot geproduceerd door Avular. Avular is een Nederlands bedrijf die specialiseert in autonome technologie de Origin is hun grond robot echter hebben ze ook een drone die autonoom kan rondvliegen. Mijn eerste doel was om te achterhalen hoe ik de Origin One autonoom kon laten rondrijden. Dit was een ontzettend interessant proces en heeft mij veel geleerd over beide autonome technologie en Lidar sensoren. De Origin was gekozen voor gebruik in dit project was omdat er al een autonome functie in zit verder waren er al 2 Origins aanwezig aan het begin van mijn project.



Xarm6

De Xarm6 is een robotarm geproduceerd door Ufactory. Ufactory is een Amerikaans bedrijf dit bedrijf specialiseert in robotarm technologie de Xarm6 is een robotarm die over 6 assen kan draaien en speciaal is gemaakt om in open ruimtes te kunnen worden gebruikt. Dit is zichtbaar door de ronde randen. Verder heeft de robotarm een gevoelige sensor die de arm stopt op het moment dat er contact wordt gemaakt als er bijvoorbeeld iemand tegen aan stoot. Om die reden is er gekozen voor de Xarm6 voor dit project omdat het uiteindelijk vrij rond gaat rijden tussen mensen is het handig als de robot zich zelf uit kan zetten als het tegen iets of iemand aan botst.



Jetson Orin Nano super

De Jetson Orin Nano super is een ontzettend nieuwe mini controller. Deze wordt geproduceerd door Nvidia, een bedrijf dat vooral bekend is voor computeronderdelen. Dit computertje hebben ze gemaakt om mensen thuis ook een AI te kunnen maken. De Jetson heeft meerdere processors, dit zorgt ervoor dat het ontzettend efficiënt AI kan hosten. Hier kan je denken aan Large Language Models (zoals chatGPT of Gemini) of beeldherkenning models, dit is waar de Jetson voor wordt gebruikt. Het zal via een camera op de robotarm het afval identificeren. Verder zal het de communicatie tussen de Origin one en Xarm6 vastleggen. De Origin heeft zelf ook 2 Jetsons. Deze zorgen voor de autonome kant van de Origin, er zou ook in deze Jetsons kunnen worden geprogrammeerd, alleen wordt dit niet geadviseerd omdat het kan riskeren dat de functies van de Origin niet meer werken. Voor deze reden is er een externe Jetson in gebruik genomen voor dit project.



Verloop project

In dit stuk zal ik omschrijven hoe ik zelf het project vond verlopen. Hier zal ik onder andere de code en tekeningen omschrijven en uitleggen waarom sommige keuzes zijn gemaakt.

Jetson starten

Obstakel 1

Met het starten van de Jetson waren er een aantal problemen. Dit was omdat het een nieuwe Jetson was dit betekende dat de OS nog op de Jetson moest worden gezet. Vaak wordt dit gedaan op een micro SD echter hadden wij gekozen om dit op een SSD-opslag te doen. Deze keuze was gemaakt omdat de SSD een veel snellere communicatiesnelheid heeft en dus zal de beeld herkenning AI sneller werken. Hierdoor was er wel wat moeite met de OS op de Jetson te zetten. Eerst had ik geprobeerd met een externe SSD cloning station de OS er op te zetten met BalenaEtcher. BalenaEtcher is een veel gebruikte app om OS op een computer te flashen. Flashen is het process van een OS-image op een opslag te zetten. Een OS-image is nodig om een computer te laten werken. Nvidia heeft hiervoor een redelijk overzichtelijke bibliotheek gemaakt waar je de OS-image van de Jetson kan vinden. Echter werkte deze manier niet dus ben ik vervolgens andere manieren gaan onderzoeken.

Obstakel 2

Na 2 dagen onderzoek had ik de Nvidia SDK gevonden. Dit is een app gemaakt door Nvidia zelf en wordt speciaal gebruikt om de OS-image op Jetsons te flashen. Hier had ik echter nogsteeds wat problemen. Er waren genoeg handige tutorials over hoe je de software op een Jetson kan zetten alleen hadden geen van deze tutorials verteld dat een Linux computer nodig is om het op hun manier te doen. Dit lukte voor mij dus niet omdat ik een Windows computer heb. Hierdoor ben ik verder gaan zoeken naar een oplossing.

Oplossing

Tijdens het onderzoeken naar een 3de oplossing bleef ik terug kijken op de Nvidia-SDK hier zat namelijk een functie om de OS-image te flashen op de NVME manier. NVME is de manier nodig om de OS-image op een SSD te zetten. Dit deed ik ook in het 2de stuk alleen kwam telkens dezelfde fout naar voren. Deze fout gaf aan dat een paar onderdelen niet konden worden verstuurd omdat ze onstabiel waren. Toen zag ik ineens een knopje dat de optie gaf om instabiele functies over te slaan. Op dit punt was ik al anderhalve week verder en was ik aan het overwegen dit gewoon op een micro-SD te proberen. Omdat ik zo laag in hoop zat op dit punt dacht ik ik probeer het gewoon. En het werkte. Ik kon de Jetson opstarten en kreeg beeld op een monitor. Vervolgens moest ik wel handmatig de inlog gegevens invullen en de onderdelen die waren overgeslagen te installeren. Dit vond ik helemaal niet erg omdat het eindelijk werkte.

Codering

Direct nadat de Jetson werkte had ik VScode geïnstalleerd dit is een app die veel wordt gebruikt om codes te schrijven en dus had ik gekozen voor VScode om mijn code te schrijven. Als eerst wilde ik in C++ beginnen met coderen omdat dit een bekende codetaal is voor mij en dus zal het voor mijn idee sneller zijn. Echter kwam ik er snel achter dat ik over moest stappen naar Python. Dit omdat ik als eerste een beeld herkenning AI wilde maken hiervoor wilde ik gebruik maken van Pytorch omdat het een betrouwbare AI training model is. Verder maakte ik gebruik van YoloV5 dit is een AI beeld herkenning manier Yolo staat voor You Only Look Once. Yolo maakt ook gebruik van Pytorch en dus kon ik niet er omheen om in Python te gaan coderen.

Voor het coderen heb ik op het moment 2 codes. Code 1 wordt gebruikt om de beeld herkenning te testen. De tweede code wordt gebruikt om communicatie tussen de Jetson en Xarm6 te testen. Hier onder volgen de 2 codes met een korte omschrijving.

Code 1: beeld herkenning

Als eerste begint de code met het importeren van verschillende bibliotheken en packages. Deze zijn nodig voor de werking van verschillende onderdelen

Vervolgens geeft je aan welk YOLO-model je wilt gebruiken. Dit wordt gedaan met de lijn “model = YOLO(‘yolov8n.pt’)” in dit voorbeeld wordt er een voorbeeld model gebruikt maar de yolov8n.pt kan worden vervangen door de naam van je eigen model.

Daarna geef je aan welke camera je wilt gebruiken. Meestal zit deze op 0 maar in dit geval stond de camera op 1 zoals te zien in “cap = cv2.VideoCapture(1)”

In een eerdere versie van de code was de FPS (Frames Per Second) redelijk laag dit wilde ik verbeteren en hiervoor heb ik de lijn “try: cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)” dit veranderde de FPS van 0.33 naar 3. Als laatste heb je hier ook nog de frame dit geeft aan hoe groot het scherm is waarmee we mee kunnen kijken. Deze is even groot als de hoeveelheid pixels is dat de camera geeft.

```
1 import time
2 import threading
3 import queue
4 import numpy as np
5 import cv2
6 from ultralytics import YOLO
7 import torch
8
9 # Choose GPU if available for much faster inference
10 device = 'cuda' if torch.cuda.is_available() else 'cpu'
11 model = YOLO('yolov8n.pt')
12 model.to(device)
13 # Fallback control and error throttling
14 tried_cpu_fallback = False
15 error_last_print = 0.0
16 ERROR_PRINT_INTERVAL = 5.0
17
18 # Camera index (adjust if needed)
19 cap = cv2.VideoCapture(1)
20
21 # Try to reduce internal capture buffering (may depend on backend)
22 try:
23     cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)
24 except Exception:
25     pass
26
27 # Small shared state and queues: keep only the latest frame/result
28 frame_lock = threading.Lock()
29 latest_frame = {'img': None}
30 result_queue = queue.Queue(maxsize=1)
31 stop_event = threading.Event()
32 # Shared statistics for terminal output
33 stats_lock = threading.Lock()
34 stats = {
35     'last_latency': 0.0,
```

De code loopt verder met het klaarzetten van een aantal waarden die vervolgens gaan worden gebruikt om gegevens te verzamelen

Daarna kom je bij de eerste loop. Dit is de capture loop. Zoals de naam zegt wordt er in deze loop de camera beelden verzameld en geprojecteerd.

In de inference loop wordt al het zware werk gedaan. Hier wordt bijvoorbeeld de FPS verhoogd en wordt er gekeken naar de snelheid van frames ook worden frames overgeslagen in de beeld herkenning als deze hetzelfde zijn. Daarbij maakt het ook annotated images. Dit zijn punten op het scherm dat laten zien welk object het ziet waar dit object wordt gezien en hoe zeker het is dat het object is wat het denkt.

```
36     'fps': 0.0,
37     'last_num_dets': 0,
38     'last_classes': []
39 }
40
41
42 def capture_loop():
43     # Drain a few frames to reduce startup lag
44     for _ in range(5):
45         cap.grab()
46     while not stop_event.is_set():
47         ret, frame = cap.read()
48         if not ret:
49             time.sleep(0.01)
50             continue
51         with frame_lock:
52             latest_frame['img'] = frame
53             time.sleep(0) # yield
54
55
56 def inference_loop():
57     global device, model, tried_cpu_fallback, error_last_print
58     last_input = None
59     while not stop_event.is_set():
60         # copy the latest frame to avoid blocking the capture thread
61         with frame_lock:
62             frame = latest_frame['img'].copy() if latest_frame['img'] is not None else None
63             if frame is None:
64                 time.sleep(0.01)
65                 continue
66
67         # If the same frame was already processed, skip
68         # (optional micro-optimization)
69         if last_input is not None and frame is last_input:
70             time.sleep(0.001)
71             continue
72
73         # Run model (this is the expensive blocking call)
74         try:
75             t0 = time.time()
76             results = model(frame, device=device)
77             t1 = time.time()
78             latency = t1 - t0
79             annotated = results[0].plot() if results and results[0] is not None else frame
80
81         # gather detection summary robustly
82         num_dets = 0
83         classes = []
84         try:
85             res0 = results[0] if results and len(results) > 0 else None
86             boxes = getattr(res0, 'boxes', None)
87             if boxes is not None:
88                 # Try length first
89                 try:
90                     num_dets = len(boxes)
91                 except Exception:
92                     # fallback to xyxy tensor length
93                     xyxy = getattr(boxes, 'xyxy', None)
94                     if xyxy is not None:
95                         try:
96                             num_dets = int(xyxy.shape[0])
97                         except Exception:
98                             num_dets = 0
99
100         # get class indices if available
101         cls_tensor = getattr(boxes, 'cls', None)
```

In het volgende stukje verzamelt het alle statistieken dit is te zien onder het stukje “with stats_lock:” hier is te zien hoe de statistieken worden verzameld en vervolgens worden laten zien. Het visualiseren van de statistieken wordt gedaan met hulp van de ingebouwde terminal waar het de gegevens print in de volgorde zoals te zien in de “print” commando’s.

Als er een fout wordt gezien zet het de statistieken allemaal op 0 zodat en wordt er een error code geprint zodat de gebruiker wel iets ziet gebeuren. Verder kan er ook een fout

gebeuren met het uitlezen van de camera beelden. Hiervoor is ook een error code en wordt de meest recente afbeelding laten zien tot de error is opgelost. Tijdens deze error gaat de code zelf proberen het probleem op te lossen. Dit doet het tot het probleem is opgelost of de code wordt gestopt.

Vervolgens doet het nog een paar achtergrond taken voordat het over loopt in de main loop.

```

102         if cls_tensor is not None:
103             try:
104                 cls_list = cls_tensor.tolist() if hasattr(cls_tensor, 'tolist') else list(cls_tensor)
105             except Exception:
106                 cls_list = []
107             # map to names if available
108             names = getattr(model, 'names', None)
109             if names:
110                 classes = [names[int(c)] for c in cls_list]
111             else:
112                 classes = [str(int(c)) for c in cls_list]
113     except Exception:
114         num_dets = 0
115         classes = []
116
117     # update stats with a small smoothing for FPS
118     with stats_lock:
119         stats['last_latency'] = latency
120         instant_fps = 1.0 / latency if latency > 0 else 0.0
121         stats['fps'] = stats['fps'] * 0.9 + instant_fps * 0.1
122         stats['last_num_dets'] = num_dets
123         stats['last_classes'] = classes
124
125     # print a concise terminal line (limit class list)
126     try:
127         unique_classes = sorted(set(classes))
128         if len(unique_classes) > 5:
129             cls_summary = ','.join(unique_classes[:5]) + '...'
130         else:
131             cls_summary = ','.join(unique_classes) if unique_classes else 'none'
132         print(f"Infer: {latency*1000:.1f} ms | FPS: {stats['fps']:.2f} | Dets: {num_dets} | Classes: {cls_summary}", flush=True)
133     except Exception:
134         print("Infer: <logging error>", flush=True)
135 except Exception as e:
136     # ensure we always emit something to the terminal so user sees activity
137     with stats_lock:
138         stats['last_latency'] = 0.0
139         stats['last_num_dets'] = 0
140         stats['last_classes'] = []
141     now = time.time()
142     # If this looks like a missing engine / CUDA/TensorRT issue, try CPU fallback once
143     msg = str(e)
144     should_fallback = ('engine' in msg.lower() or 'cuda' in msg.lower() or 'tensorrt' in msg.lower()) and not tried_cpu_fall
145     if should_fallback and device != 'cpu':
146         print("Infer: detected engine error attempting to fall back to CPU (this may be slower)", flush=True)
147         try:
148             device = 'cpu'
149             model.to('cpu')
150             tried_cpu_fallback = True
151             print("Infer: fallback to CPU complete", flush=True)
152         except Exception as e2:
153             print(f"Infer: CPU fallback failed: {type(e2).__name__} {str(e2)[:200]}", flush=True)
154     else:
155         # rate-limit error printing to avoid terminal spam
156         if now - error_last_print > ERROR_PRINT_INTERVAL:
157             print(f"Infer: ERROR during model call: {type(e).__name__} - {msg[:200]}", flush=True)
158             error_last_print = now
159     annotated = frame
160
161     # Keep only the latest annotated frame
162     try:
163         result_queue.put_nowait(annotated)
164     except queue.Full:
165         try:
166             _ = result_queue.get_nowait()
167         except queue.Empty:
168             pass
169         try:
170             result_queue.put_nowait(annotated)
171         except queue.Full:
172             pass
173
174     last_input = frame
175
176
177 # Start background threads
178 cap_thread = threading.Thread(target=capture_loop, daemon=True)
179 inf_thread = threading.Thread(target=inference_loop, daemon=True)
180 cap_thread.start()
181 inf_thread.start()
182
183
184 def main_loop():
185     last_shown = None
186     last_status_time = 0.0
187     while True:
188         # Prefer showing annotated frames when available
189         try:
190             annotated = result_queue.get_nowait()
191             last_shown = annotated
192         except queue.Empty:
193             with frame_lock:

```

In de main loop zet laat het een tablad zien waar alles visueel te zien is. In dit stuk wordt ook naar de andere loops geroepen. Dit activeert de andere loops en dit stuk bepaald dus ook of de code doorgaat. Door CTRL+C in te drukken stop je de main loop en gaat de code helemaal onderaan door naar de “finally” hier wordt het YOLO-model gestopt samen met de camera informatie. Nadat alles veilig is stopgezet wordt de code afgesloten.

```
187 while True:
188     # Prefer showing annotated frames when available
189     try:
190         annotated = result_queue.get_nowait()
191         last_shown = annotated
192     except queue.Empty:
193         with frame_lock:
194             raw = latest_frame['img'].copy() if latest_frame['img'] is not None else None
195             if raw is not None and last_shown is None:
196                 last_shown = raw
197
198     if last_shown is not None:
199         cv2.imshow('trashcam', last_shown)
200     # Also print periodic capture/display stats (once per second) to help debugging
201     now = time.time()
202     if now - last_status_time > 1.0:
203         with stats_lock:
204             lat = stats['last_latency']
205             fps = stats['fps']
206             nd = stats['last_num_dets']
207             print(f"status {lat} latency: {lat*1000:.1f} ms | model FPS: {fps:.2f} | last detections: {nd}")
208             last_status_time = now
209         if cv2.waitKey(1) & 0xFF == ord('q'):
210             break
211
212
213 if __name__ == '__main__':
214     try:
215         main_loop()
216     finally:
217         stop_event.set()
218         time.sleep(0.05)
219         cap.release()
220         cv2.destroyAllWindows()
```

Code 2: pin signal

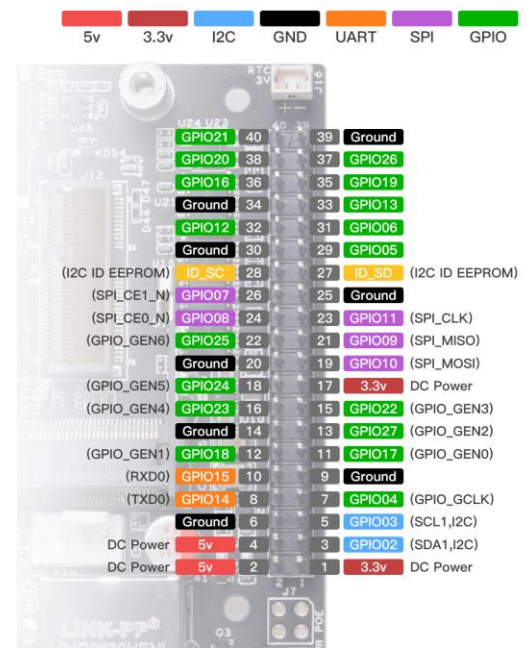
Als eerste wordt in deze code weer de bibliotheken en packages bij elkaar gehaald.

Vervolgens maak ik een lijst van pins en geef ik de pins een naam. De pins benoem je met een GPIO-bibliotheek. Deze bibliotheek zorgt er alleen wel voor dat de pins een andere naam krijgen dan hoe ze op de Jetson heten. Hiervoor heb ik een handige foto gevonden die mij veel heeft geholpen met de juiste GPIO-naam bij de juiste pin. Deze foto heb ik onder dit stukje code bijgevoegd. Er zijn 2 (3) soorten pins in de lijst. De set pins sturen signalen uit zoals te zien bij GPIO.setup. De in pins worden gebruikt om signalen te ontvangen. Als laatste heb je een derde pin soort de rst pin rst staat hier voor reset en wordt gebruikt om een signaal uit te sturen om de locatie van de arm terug naar de start locatie te zetten.

Verder zet ik alle uitgaande signalen standaard op 0 zodat er niet perongeluk een pin open staat en de arm een onverwachte beweging maakt.

```
1 import os
2 import Jetson.GPIO as GPIO
3 import time
4 import keyboard
5
6 # Pin Definitions
7 set1 = 17 # BOARD pin 11, BCM pin 17
8 set2 = 18 # BOARD pin 12, BCM pin 18
9 set3 = 27 # BOARD pin 13, BCM pin 27
10 set4 = 22 # BOARD pin 15, BCM pin 22
11
12 in1 = 10 # BOARD pin 19, BCM pin 10
13 in2 = 9 # BOARD pin 21, BCM pin 9
14 in3 = 25 # BOARD pin 22, BCM pin 25
15 in4 = 11 # BOARD pin 23, BCM pin 11
16
17 rst = 8 # BOARD pin 24, BCM pin 8
18
19 # Pin Setup:
20 GPIO.setmode(GPIO.BCM) # BCM pin-numbering scheme
21 GPIO.setup(set1, GPIO.OUT) # set1 pin set as output
22 GPIO.setup(set2, GPIO.OUT) # set2 pin set as output
23 GPIO.setup(set3, GPIO.OUT) # set3 pin set as output
24 GPIO.setup(set4, GPIO.OUT) # set4 pin set as output
25
26 GPIO.setup(in1, GPIO.IN)
27 GPIO.setup(in2, GPIO.IN)
28 GPIO.setup(in3, GPIO.IN)
29 GPIO.setup(in4, GPIO.IN)
30
31 GPIO.setup(rst, GPIO.OUT) # rst pin set as output
32
33 # Initial state for LEDs:
34 GPIO.output(set1, GPIO.LOW)
35 GPIO.output(set2, GPIO.LOW)
```

Hiernaast zie je de GPIO-pin list het binnenste stuk laat zien welke bord pin het is en de buitenste laat de naam van de bord pin zien. Sommige pins hebben nog extra tekst ernaast staan deze tekst vertelt of het voor een specifieke toepassing kan worden gebruikt.



Vervolgens nadat alle setpins naar 0 zijn gezet begint de code. Hier zie je eerst dat het zegt dat de code gaat beginnen en als je de code wilt stoppen Q moet drukken. Je ziet hier dat er een “while true:” loop wordt gebruikt. Deze zal worden vervangen met de main loop als je het zal combineren met code 1 dit is hier in de test niet nodig. Als eerste checkt het alle input pins of de arm beschikbaar is om signalen te ontvangen. Als het beschikbaar is om signalen te ontvangen gaat de code door om te kijken of er een toets op het toetsen bord wordt ingedrukt. Als

```

36 GPIO.output(set3, GPIO.LOW)
37 GPIO.output(set4, GPIO.LOW)
38 GPIO.output(rst, GPIO.LOW)
39
40 print("Starting demo now! Press q to exit")
41 try:
42     while True:
43         if GPIO.input(in1) == GPIO.LOW and GPIO.input(in2) == GPIO.LOW and GPIO.input(in3) == GPIO.LOW:
44             if keyboard.is_pressed('1'):
45                 GPIO.output(set1, GPIO.HIGH)
46                 print("state 1 started")
47                 wait_time = 0
48                 while wait_time < 5: # Wait for 5 seconds or until '0' is pressed
49                     if keyboard.is_pressed('0'):
50                         print("State 1 interrupted")
51                         break
52                     time.sleep(0.5)
53                     wait_time += 0.5
54                 GPIO.output(set1, GPIO.LOW) # Turn off after wait
55             elif keyboard.is_pressed('2'):
56                 GPIO.output(set2, GPIO.HIGH)
57                 print("state 2 started")
58                 wait_time = 0
59                 while wait_time < 5: # Wait for 5 seconds or until '0' is pressed
60                     if keyboard.is_pressed('0'):
61                         print("State 2 interrupted")
62                         break
63                     time.sleep(0.5)
64                     wait_time += 0.5
65                 GPIO.output(set2, GPIO.LOW) # Turn off after wait
66             elif keyboard.is_pressed('3'):
67                 GPIO.output(set3, GPIO.HIGH)
68                 print("state 3 started")
69                 wait_time = 0
70                 while wait_time < 5: # Wait for 5 seconds or until '0' is pressed

```

bijvoorbeeld “1” wordt ingedrukt dan wordt set1 actief. Dit signaal gaat dan de robotarm bewegen. Verder is er ook een interrupt optie om het signaal te verbreken stel je hebt het verkeerde signaal gestuurd.

In het laatste stuk worden alle set signalen naar 0 gehaald om weer te voorkomen dat een signaal aan blijft staan.

Ook wordt er gekeken of q is ingedrukt. Is dit ingedrukt dan zegt de code dat de code wordt gestopt wordt rst aangezet en na een korte seconde gaat het naar finally waar de GPIO wordt geleegd. Dit zorgt ervoor dat de pins worden ge deactiveert.

```
71         if keyboard.is_pressed('0'):
72             print("State 3 interrupted")
73             break
74         time.sleep(0.5)
75         wait_time += 0.5
76         GPIO.output(set3, GPIO.LOW) # Turn off after wait
77     elif keyboard.is_pressed('4'):
78         GPIO.output(set4, GPIO.HIGH)
79         print("state 4 started")
80         wait_time = 0
81         while wait_time < 5: # Wait for 5 seconds or until '0' is pressed
82             if keyboard.is_pressed('0'):
83                 print("State 4 interrupted")
84                 break
85             time.sleep(0.5)
86             wait_time += 0.5
87             GPIO.output(set4, GPIO.LOW) # Turn off after wait
88     elif keyboard.is_pressed('0'):
89         print("no state started")
90         GPIO.output(set1, GPIO.LOW)
91         GPIO.output(set2, GPIO.LOW)
92         GPIO.output(set3, GPIO.LOW)
93         GPIO.output(set4, GPIO.LOW)
94     time.sleep(0.1) # Small delay to debounce key presses
95     if keyboard.is_pressed('q'):
96         print("Exiting demo.")
97         GPIO.output(rst, GPIO.HIGH) # Set reset pin high to reset the system
98         time.sleep(1) # Wait for a moment to ensure the reset signal is registered
99         break
100 finally:
101     GPIO.cleanup() # Clean up GPIO on exit
```

Obstakel code 2

Tijdens het maken en testen van de tweede code kwam ik een probleem tegen met het aansturen van de pins. Hiervoor moest ik gebruik maken van de GPIO-pins zoals eerder aangegeven. Hiervoor heb ik redelijk veel moeite gehad en had ik eerst via Nvidia zelf een oplossing gevonden voor de Jetson. Dit was alleen ontzettend veel werk. Toch was ik er eerst mee begonnen tot ik uiteindelijk dacht dat het wel makkelijker zal kunnen. Na nog verder zoeken kon ik eindelijk een oplossing vinden hiervoor. Deze oplossing was veel makkelijker en sneller dan de eerdere oplossing en had ik gevonden met behulp van een simpele video op YouTube de volgende link stuurt je naar de video: [GPIO fix](#)

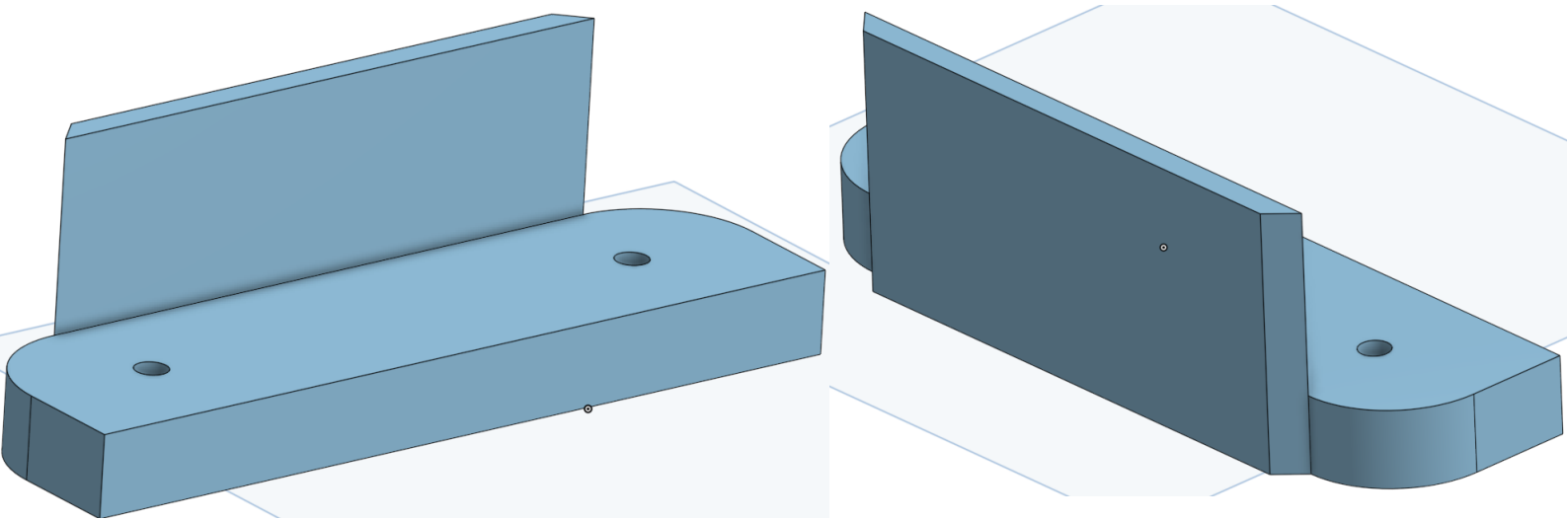
De tutorial is eigenlijk al klaar na 1:50 echter raad ik wel aan om verder te kijken om zeker te weten dat de installatie succesvol is.

Tekeningen

In dit stuk zal ik de tekeningen bespreken. Hier ga ik het hebben over beide de eind tekeningen en de tekeningen die niet door zijn gegaan.

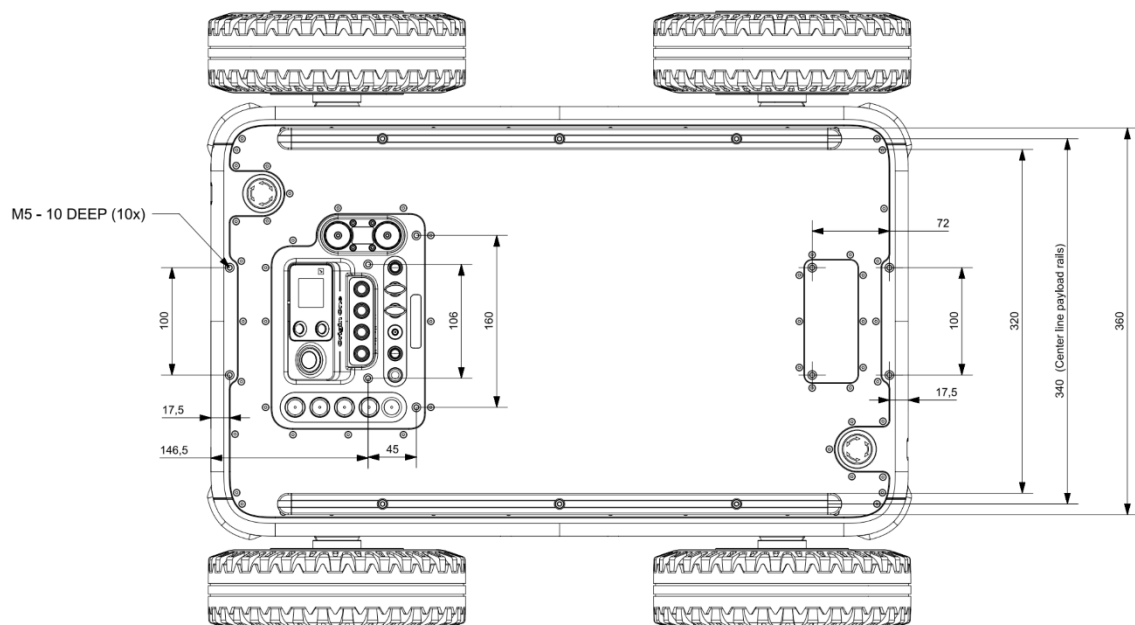
3d tekeningen

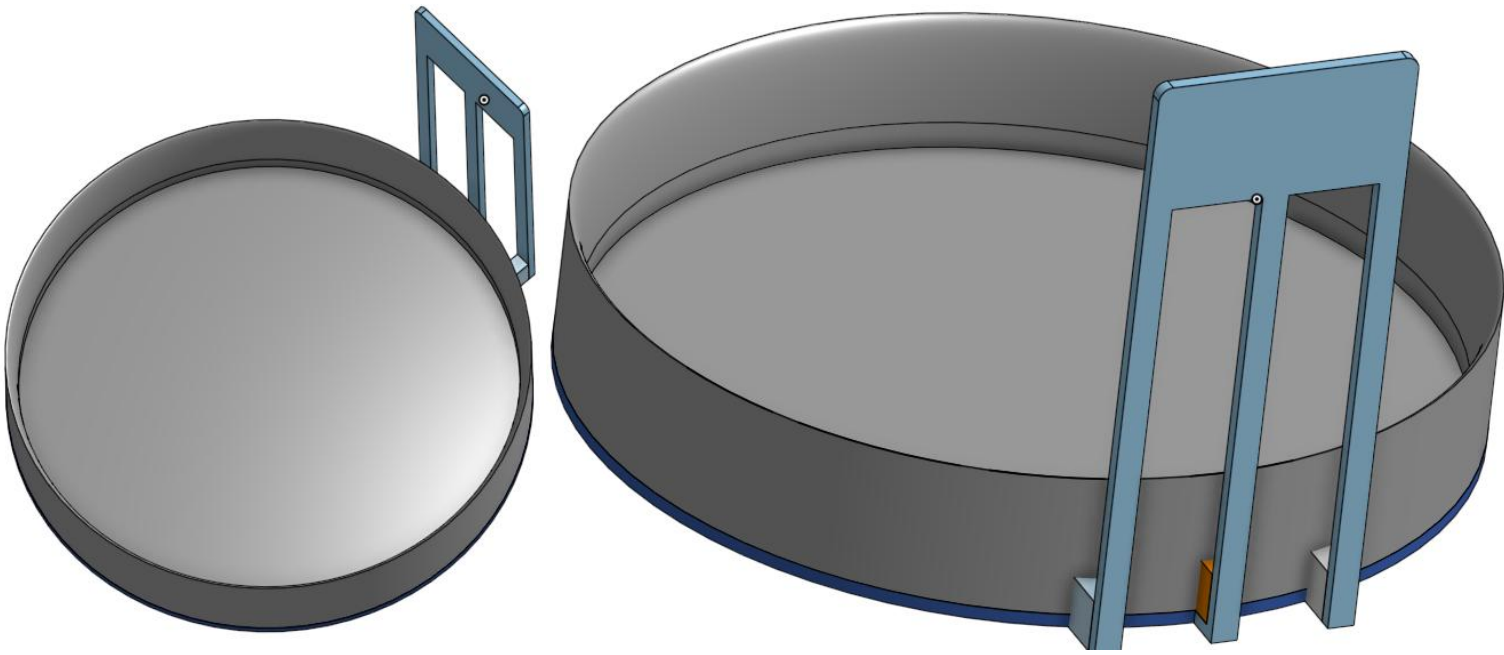
Voor dit project zijn er verschillende 3D tekeningen gemaakt. Deze waren nodig voor de prullenbak behuizing die achter de Origin komt te hangen. Hier is gebruik gemaakt van de avular github pagina en zijn er zelf afmetingen gemaakt.



Can connect (Origin side)

Als eerste hebben we de connector gemaakt die op de origin komt zitten. We hebben besloten het niet 1 geheel te maken omdat het makkelijk moet worden los gemaakt. Aan de platte kant die omhoog staat wordt klittenband geplakt om de prullenbak vast te zetten. Hier voor zijn foto's van de Avular github pagina gebruikt deze foto is hier onder te zien. Deze foto is gebruikt om de afstand tussen de 2 schroef gaten en de gat grote vast te stellen.





Can Connect (Can side)

Voor deze tekening moest er worden besloten welke prullenbak er moest worden gebruikt. Hier was ik beland op de uiteindelijke keuzen van een prullen bak van 18L en 31 x 32 cm. Ik heb deze prullenbak gekozen omdat het geen deksel heeft en dus hoeft de robotarm niet de prullenbak open te trekken. Verder is het niet een te grote prullenbak en zal het er niet apart uitzien. Ook is de prullenbak toch niet te klein dat het elk half uur zal moeten worden geleegd. Verder is hier weer een plat stuk om de klittenband aan vast te zetten en is er ruimte overgehouden om mogelijk ook nog wielen onder de houder te zetten als er meer versteviging nodig is. Als laatste is de binnen kant van de houder een stukje breder gemaakt zodat de prullen bak nog speling heeft en makkelijk er uit te halen is en weer makkelijk erin kan worden gezet.

Klittenband

Ik heb gekozen om klittenband te gebruiken om de 2 onderdelen vast te zetten. Dit omdat het gaat worden gebruikt in mogelijk drukke plekken en heeft daar door de kans om een keer een stoot te krijgen. Hiervoor willen we voorkomen dat er iemand gewond raakt of het mechanisme kapotgaat. De klittenband zorgt er voor dat het los komt als er een stoot tegen aan wordt gegeven.

Elektrotechnische tekeningen

Voor dit project zijn meerdere elektrotechnische tekeningen gemaakt door mij. Hiervoor heb ik gebruikgemaakt van Qelektrotech en KiCad.

Belangrijke termen

In dit korte stuk zal ik snel een paar belangrijke termen uitleggen.

PCB

PCB staat voor Printed Circuit Board. Dit vertaald naar geprinte schakeling bord. Deze worden in bijna alle elektrische applicaties. De borden kun je bijvoorbeeld in je afstands bediening vinden als je deze open maakt. De bordjes zijn altijd groen met een paar uitzonderingen.

Matrix

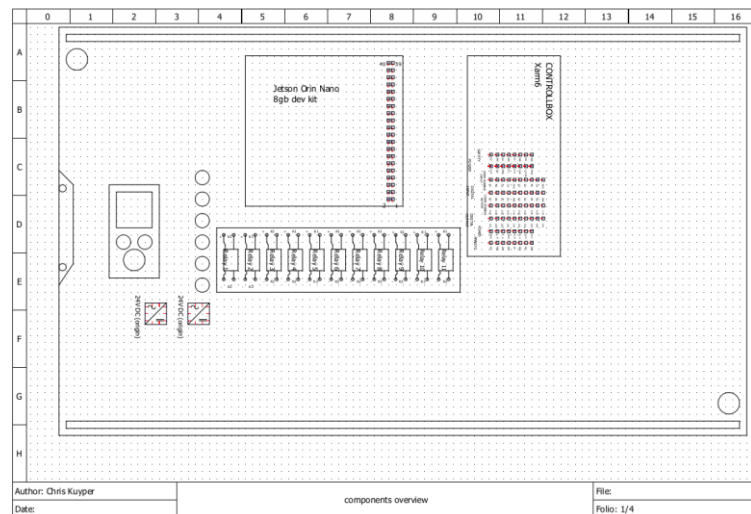
Een matrix in de elektrotechniek wijst altijd naar een punt waar meerdere van dezelfde soort schakelingen samen zitten. Dit zit vaak op een PCB dit wordt vaak gebruikt om de algemene schakeling overzichtelijk te houden. Ook maken matrix's het makkelijk om een probleem op te sporen.

Qelektrotech tekeningen

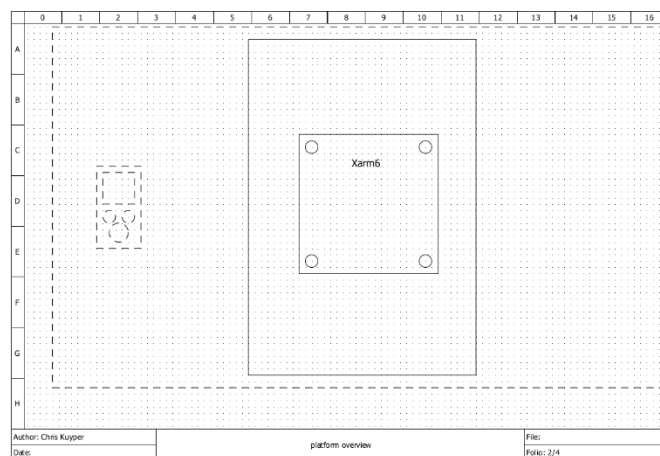
Quelektrotech is een app die wordt gebruikt om de tekeningen te maken voor onder andere elektrische kasten. Ik heb het hier gebruikt om een tekening te maken voor een overzicht van de onderdelen en een grove locatie van alle onderdelen.

V1

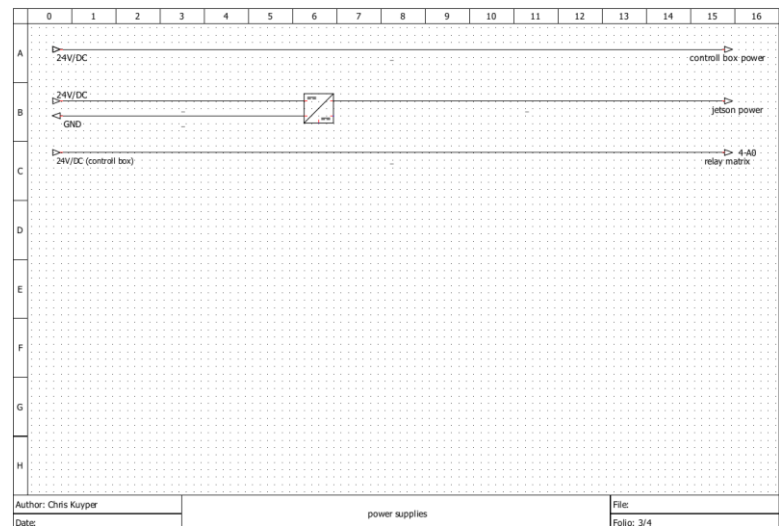
Folio 1: op deze pagina/folio is de origin getekend en is de jetson orin nano en controll box van de Xarm6 zichtbaar. Deze tekening is niet op maat getekend en is vooral zodat het zichtbaar is welke onderdelen er zijn. Ook is de Relay Matrix zichtbaar. Deze was er voor een korte tijd tussen uit gegaan maar is weer terug echter ziet het er niet zo uit als op deze foto zichtbaar.



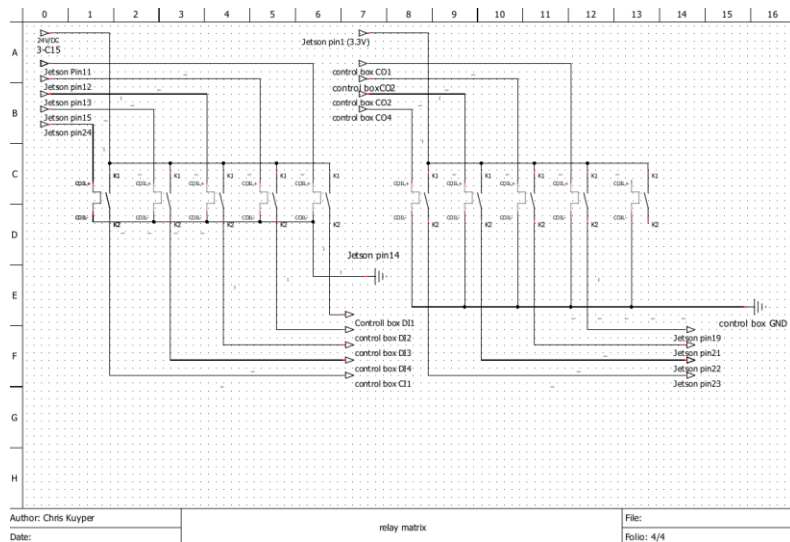
Folio 2: op deze pagina is de origin weer getekend echter is deze getekend met stippellijntjes dit heb ik gedaan om nadruk te leggen dat deze pagina richt op dingen die hoger zitten dan de origin. Hier is te zien dat het alleen de Xarm6 op deze hoogte zit en verder niks



Folio 3: op dit folio is de stroom voeding te zien. Hier zie je dat de bovenste lijn naar de controll box van de Xarm6 gaat. De tweede gaat naar de jetson hiervoor zal een stepdown converter moeten worden toegevoegd gezien de jetson 19V moet krijgen en niet 24V. Verder zie je dat de relay matrix 24V krijgt van de controll box dit omdat de matrix communicatie verzorgt tussen de jetson en Xarm6 alleen kan de Jetson niet genoeg stroom leveren voor de controll box om optepakken.



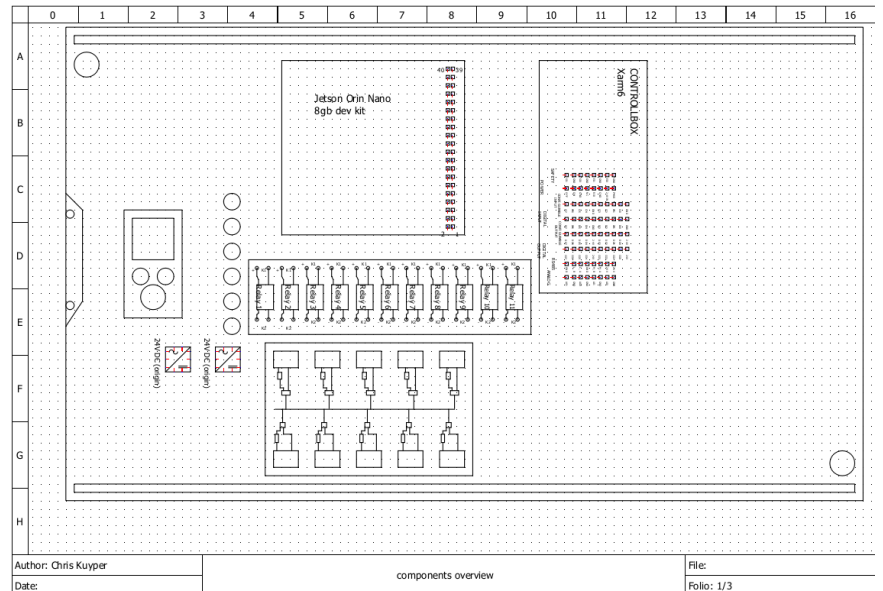
Folio 4: op deze pagina is de relay matrix uitgetekend. Hier zie je dat de linker kant de communicatie naar de controll box doet en de rechter kant de communicatie naar de jetson doet. Dit had ik gedaan omdat de controlbox signalen kan uitsturen als het een foutmelding krijgt en de jetson kan het hier dus ontvangen. Het maakt gebruik van verschillende soorten relais. Echter was ik iets te enthousiast met het tekenen van dit en waren we er later achter gekomen dat de jetson te weinig Ampere levert om de relais aan te sturen. Hiervoor zijn Timo en ik weer naar het ideeën bord gegaan en zijn weer gaan denken over een oplossing



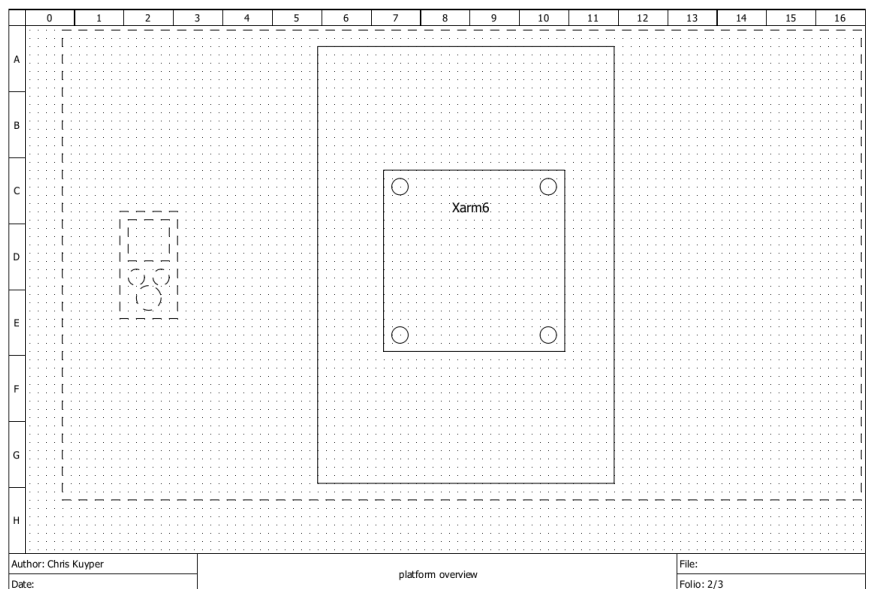
V2

deze versie is gecombineerd met een aantal PCB tekeningen van kicad deze zullen dus ook hier worden besproken

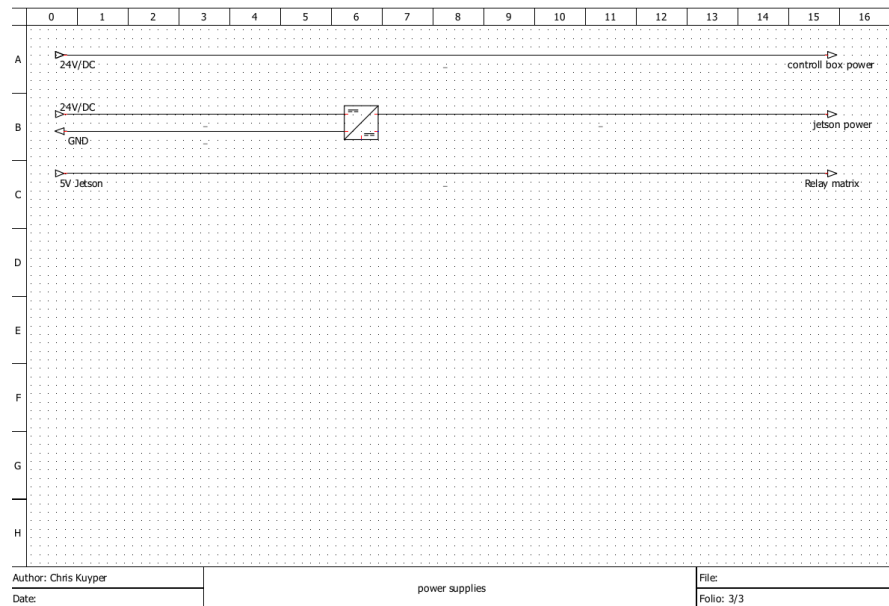
Folio 1: hier zie je de origin uitgetekend met verschillende onderdelen toegevoegd onder andere zie je de Jetson Orin Nano deze zal communicatie tussen de origin en controlbox doen. Verder zie je zoals al gezegd een controlbox voor de Xarm6. Daarbij zie je ook een Relay matrix en een transistor matrix. Deze tekeningen zijn niet op maat getekend en zullen afwijken van de uiteindelijke werkelijke grote



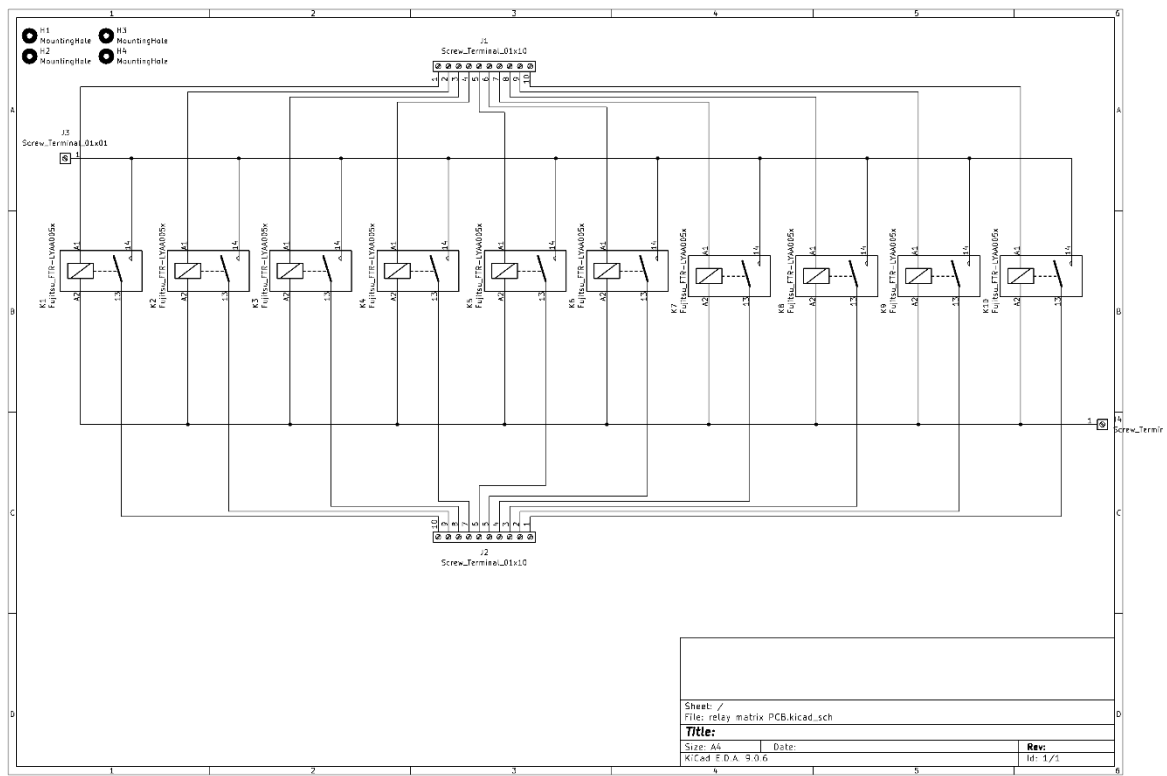
Folio 2: hier zal je geen verschil zien omdat hier duidelijk niets veranderd hier zie je nog steeds alleen de Xarm6 op de hoogte boven de Origin.



Folio 3: hier zie je de stroom loop voor de verschillende onderdelen. Hier zie je bijvoorbeeld dat er 24V direct naar de controlbox gaat. Verder kan je zien dat er 24V uit richting een DC/DC converter gaat deze veranderd de stroom naar 19V wat nodig is voor de jetson. Ook zie je dat er 5V van de jetson naar de relay matrix gaat.

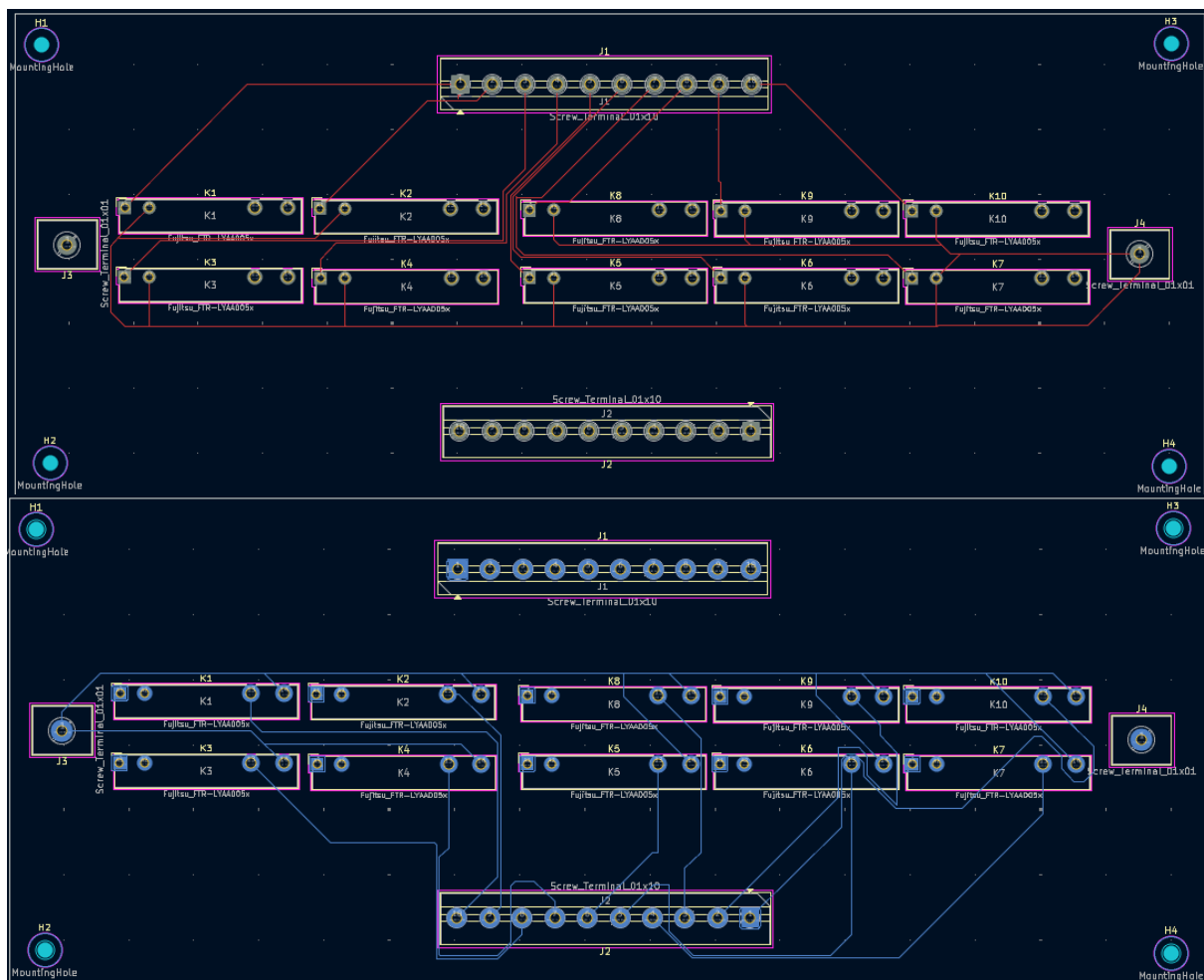


Relay matrix: de relay matrix gaat worden gebruikt om signalen van de controlbox naar de jetson te sturen. Deze is nodig omdat de controlbox 24V/dc uitstuurt en de Jetson kan die spanningen niet aan.



Elektrisch schema:

Hierboven is het elektrische schema zichtbaar. Hier zie je aan beide de boven en onderkant 2 screw terminals. Dit zijn onderdelen op een pcb waar je draden in kan schroeven om de connecties te maken. Hier is dus geen soldeer werk nodig en draden kunnen makkelijk worden uit gewisseld. Verder zijn er 2 screw terminals die maar 1 slot hebben. Dit is omdat deze de 5V van de Jetson zijn en de ground van de controlbox. Ik heb deze 2 aan de tegenovergestelde kanten gezet om verwarring in de toekomst te voorkomen.



PCB layout (relay matrix):

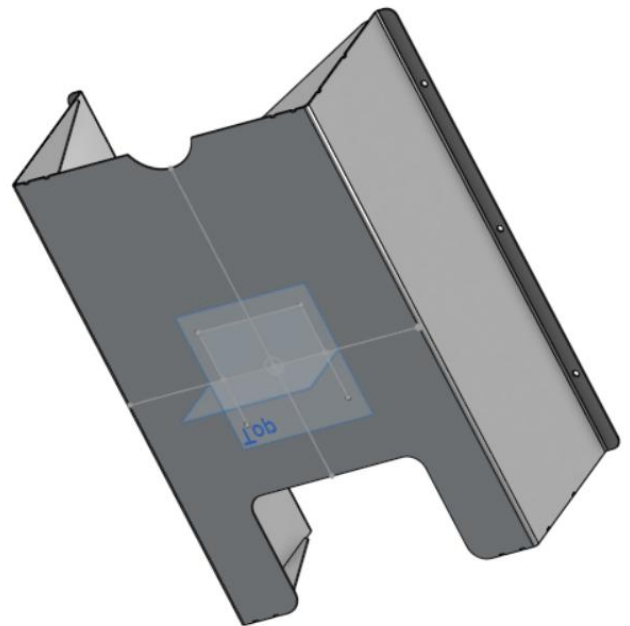
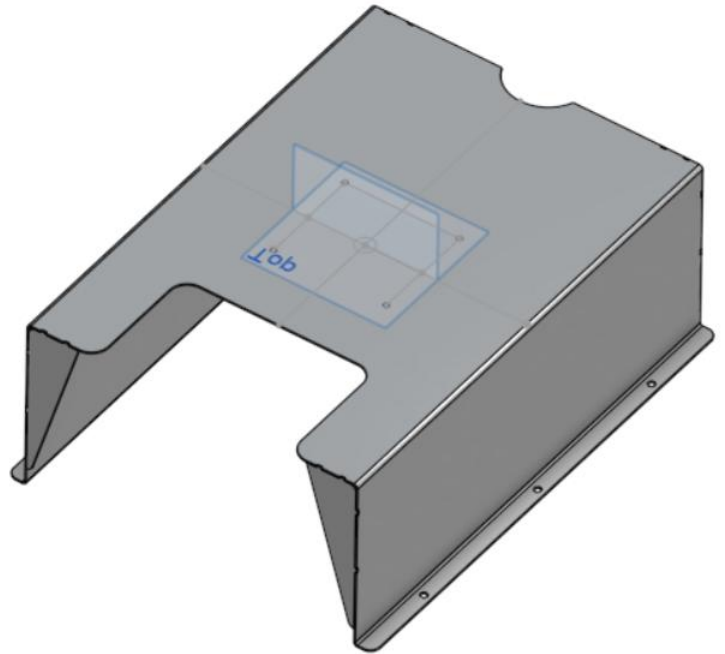
Op de 2 foto's hierboven is de layout van de PCB zichtbaar. Ik moest de pcb-bedrading verdelen over 2 kanten. Dit omdat de koperen draden niet door elkaar kunnen en dus zouden de draden elkaar blokkeren. De rode kant is de boven kant en de blauwe kant is de onderkant. Je kan zien dat er tussen de eerste twee en laatste drie relais meer ruimte is. Dit is gedaan zodat de koperdraden netjes daardoorheen kunnen. Op de rode kant is dat ontzettend mooi te zien. Ik ben ook heel tevreden met de rode kant. Echter ben ik iets minder tevreden met de blauwe kant. De draden maken soms een aparte route naar hun eindbestemming. Dit heeft temaken met hoe het elektrische schema is getekend. Ik vond beide de voorkant en het schema er netjes uitzien en was bang als ik het schema aan ging passen dat beide het schema en de voorkant er minder netjes uit gingen zien. Om die reden heb ik ervoor gekozen het te houden zoals het is (met wat tegenzin en overtuiging van Timo).

Xarm6 docking

De robotarm moet op een manier vast komen te zitten op de Origin one.

Voorheen werd dit gedaan door een onstabiele houten constructie. Dit wilde ik vervangen door een metalen plaat voor meer stevigheid en water veiligheid. Hiervoor heb ik een aantal MBO studenten voor aangestuurd.

Deze kwamen ook hier stagelopen en er was op dat moment geen project voor hen ingepland. Ik was voor hen een aanspreek punt voor vragen. Ook had ik een aantal eisen aan hen opgesteld zoals dat de constructie moest worden gemaakt met aluminium platen. Hiervoor moesten ze het laten lasersnijden hierbij moest ik ook een groot deel helpen. Ook moest de plaat boven een bepaalde hoogte zitten zodat het geen problemen veroorzaakt met de Lidar van de Origin.

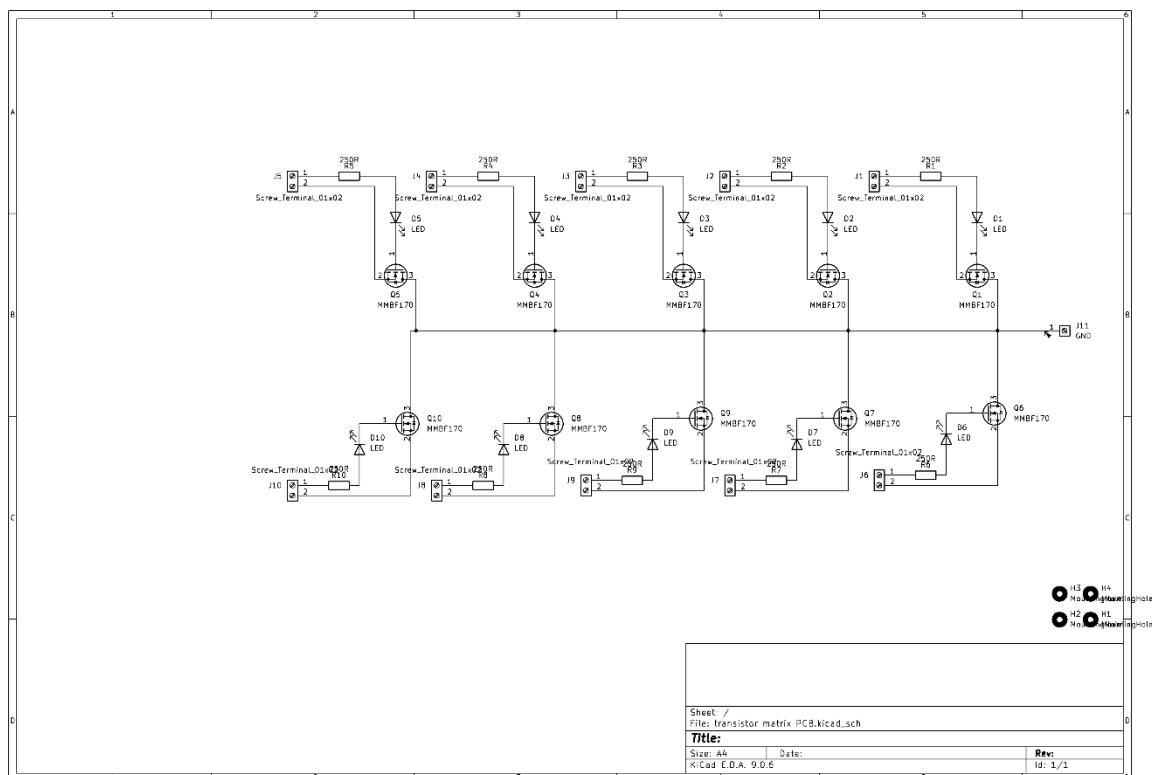


Origin docking station

Naast de plaat om de Xarm6 vast te zetten was er ook een vraag gekomen om een station te maken waar beide Origins kunnen staan als deze niet gebruikt worden. Dit project werd door dezelfde studenten uitgevoerd. Ook hier heb ik geholpen met meerdere onderdelen en vragen. Ik heb het aan hen overgelaten om te kiezen welk materiaal wordt gebruikt. Wel konden ze naar mij toekomen voor advies maar ik heb de eind keuze aan hen overgelaten. Ook was er een eis dat beide Origins boven elkaar moesten staan als ze niet worden gebruikt.

transistor matrix:

na een tijdje met Timo samen na te denken kwamen we met het idee om met een transistor te gaan werken. We mochten een aantal transistoren van een ander lab lenen en begonnen te testen. Ik begon met de Jetson direct de aansturing te testen hier kwam ik er achter dat de pins van de Jetson niet schakelde zoals het hoorde. Om dit probleem op te lossen had ik snel een video opgezocht en na het volgen van deze video werkten de pins weer. Vervolgens hebben Timo en ik nog steeds wat moeite gehad met het aansturen van de transistor dachten wij. Het bleek uiteindelijk dat de transistor correct werd aangestuurd alleen moest de input pin van de controlbox naar de GND worden geschakeld in plaats van de 24V naar de input. Dit is redelijk abnormaal voor input pins wat voor de eerdere moeite zorgde.



Transistor matrix schema:

Hierboven is het schema van de transistor matrix zichtbaar. Deze bestaat uit 10 van de zelfde schakelingen om die reden ga ik 1 van de schakelingen aflopen. Als eerste heb je de dual screw terminal, deze wordt gebruikt om draden te verbinden aan de PCB. Vervolgens zie je een weerstand van 250ohm hier stroomt de 3.3V van de Jetson Orin Nano over. Daarna heb je een LED, de kleur hiervoor maakt niet uit maar de voorkeur gaat naar een groene of gele LED, deze zorgt er ook voor dat er geen stroom van de transistor terug zal kunnen stromen. Als laatste heb je de transistor, hier komt de 3.3V van de Jetson binnen op de Base pin en de 24V binnen op de collector pin, verder komt er stroom uit de emitter pin die de stroom naar de enkele screw terminal stuurt deze zit verbonden aan de GND van de controlbox.

Vorbereidingen

Nadat alle tekeningen waren gemaakt werd het tijd om de voorbereidingen te doen. Hiervoor wilde ik een lijst maken van onderdelen die nog moeten worden besteld. Hier kwam ik er echter achter dat de onderdelen al aanwezig waren voor het grijpen. Hiervoor hoefde dus niets besteld te worden. Wel moesten er een paar dingen worden print zoals de “Can Connect” deze print was echter uitgesteld omdat het niet direct nodig was en anders in de weg zal zitten. Gezien het lab niet al te groot is en het al redelijk vol stond. Daarbij was er ook zelf een 3D printer besteld voor het lab en dus zouden we niet zelf filament moeten hebben om in het makers lab te printen. Verder werden de PCB’s niet direct geprint om eerst te achterhalen of de schema’s werken met behulp van breadboards.

Overige onderwerpen

In de volgende stukken zal ik de onderwerpen bespreken die niet tussen de loop van het project passen.

Onderhoud

Eerst was het niet verwacht dat er tijdens dit project onderhoud moest plaats vinden. Echter is dit dus anders gelopen. Dit was omdat de Origin waar ik mee werkte een paar problemen had daardoor werd mijn project regelmatig vertraagd. Voorheen had ik het probleem bij de labhoofd en mijn stagebegeleider. Hier was gezegd dat er contact zal worden opgenomen met Avular alleen zag ik een paar weken later nog niets gebeuren. Uiteindelijk besloot ik zelf het initiatief te nemen om contact op te nemen. Hiervoor heb ik eerst een mail gestuurd naar info@avular.com deze gaven een dag later een reactie dat het probleem is doorgestuurd naar de serviceafdeling. Daar kreeg ik redelijk snel een telefoontje van Martijn hij belde om wat meer informatie te krijgen over het probleem. Ik heb alles dat ik wist via de telefoon door gegeven vervolgens ging hij met de andere op de service afdeling overleggen hoe dit gaat worden opgelost. Snel daar na kreeg ik weer een telefoontje hier vertelde Martijn dat de Origin zal moeten worden opgestuurd. En legde uit wat het proces is voor het opsturen. Hier heb ik ook binnen het lab overlegd hoe we dit kunnen doen en zijn we uiteindelijk geëindigd met een idee om de robot naar een gebouw van het HVA te brengen die in de vakantie wel open was. Wij waren eerst bij het gebouw langs geweest om te vragen of het kon en dat mocht gelukkig. Vervolgens heb ik de benodigde gegevens naar Avular gestuurd en is verder alles door hun geregeld. Hier onder zijn de mails gezien die zijn uitgewisseld

Beste heer/mevrouw,

Mijn naam is Chris Kuyper, ik ben een MBO stagiair op het HVA. Voor mijn examen opdracht moet ik werken met een van de Origin one's die op het HVA staan.

Tijdens mijn examen-project ben ik iets interessants tegengekomen, het lijkt erop alsof 1 van de Origins naar een soort idle state gaat (dit is te zien in het filmpje die ik heb bijgevoegd). Er is geen vast ritme in wanneer het in deze status gaat. Het is al een paar keer gebeurd dat het naar deze status gaat terwijl we de Origin aan het besturen waren of terwijl het een autonome missie uitvoerde. Ook is in het filmpje te zien dat de noodstop niks doet met de Origin en het scherm niks laat zien.

De lichten zouden moeten indiceren dat het aan het opstarten is, dit is alleen apart gezien het naar deze status gaat nadat het is opgestart. Dit doet de andere origin echter niet. De enige oplossing die wij tot nu toe hebben gevonden is de aan/uit knop ingedrukt houden tot de Origin uit is. Het kort indrukken van de aan/uit knop doet niks. Verder hebben wij van alles geprobeerd om de Origin uit deze status te halen. Verder konden wij niks vinden dat signaleert dat dit eerder is voorgekomen.

Graag hoor ik snel van jullie of hier een oplossing voor is of dat er iemand van Avular langs moet komen om dit zelf beter vast te documenteren.

Met vriendelijke groet,
Chris Kuyper

Tel: 06 13254547

Mail 1

Chris,

Ik heb jouw vraag naar onze serviceafdeling gestuurd. Zij zullen er bij je op terugkomen.

Groet,

Ralph Broeders

Head of Sales

+31621154494

www.avular.com

[LinkedIn](#) - [Instagram](#)

Achtseweg Zuid 221
5651GW Eindhoven
The Netherlands



Avular
Building Imagination

Reactie mail 1

Hey Chris,

Zoals besproken aan de telefoon moet de origin helaas terug naar Avular om te kijken wat er mis mee is. Voor dit vragen wij aan jou om de origin in te pakken in een doos en de afmetingen door te geven aan ons. Dan zorgen wij voor een verzend label die je kan gebruiken voor de verzending.

Mochten er nog vragen zijn dan hoor ik het graag.

Met vriendelijke groet, / Best regards,

Reactie mail 2.1

Hey Chris,

Zou je de volgende informatie kunnen geven voor het ophaal adres:

Straat + nr eventueel gebouw

Postcode

De volgende dingen moeten wij hebben:

Origin

Controller

E-stop

Oplader

Met vriendelijke groet, / Best regards,

Reactie mail 2.2

Beste,

Chris Kuijper en ik hebben afgesproken dat de origin terug terug moet naar Avular, het pakket zou verzorgd worden door de receptie i.v.m vakantie.

In de bijlage vind u het verzend label, zou u dit op de kist willen bevestigen?

Fedex zal morgen (29/04) het pakket ophalen.

Mochten er vragen zijn kunt u mij bereiken via dit email adres of het telefoon nummer beschreven onder.

Met vriendelijke groet, / Best regards,

Eind mail

Zoals te zien in de mails is veel communicatie via de telefoon gedaan. Dit is gedaan omdat het een snellere en effectievere manier is om te communiceren.

Obstakels

Tijdens dit project ben ik tegen veel obstakels opgelopen hieronder ga ik vast stellen welke obstakels en hoe ik deze heb opgelost of hoe ik met een omweg dit heb geregeld. Een van de grootste obstakels heb ik net al besproken dit was namelijk de Origin. Ook heb ik het al gehad over de moeite aan het begin met de Jetson klaar maken. Dit kwam voor een groot deel doordat er niemand aanwezig is/was die kennis heeft met de Jetson. Jammer genoeg zijn er echter nog een aantal obstakels geweest die ik hier nog ga vertellen.

Te weinig kennis

Over het algemeen heeft de kennis mij hier redelijk in de steek gelaten. Zelf had ik weinig kennis in veel systemen en er waren geen mensen aanwezig die de kennis hadden die ik nodig had. Dit zorgde ervoor dat ik soms een week lang moest onderzoeken voor een onderdeel. Ook hielp het niet dat de systemen die ik moest gebruiken zo nieuw zijn. Hierdoor waren er vaak onduidelijke of incorrecte tutorials.

ROS2

ROS2 staat voor Robot Operating System. Dit is nodig om commando's naar de Origin te sturen en berichten van de Origin af te lezen. Jammer genoeg wordt dit vaak op desktops gebruikt die op Linux werken. Ik had voordat ik begon te werken met de Jetson geen ervaring met Linux en er waren geen mensen die ervaring hadden met ROS2 beschikbaar. Verder omdat er geen of weinig tutorials zijn om ROS2 op een Jetson te installeren was het een enorme puzzel om het werkend te krijgen. Ik kon namelijk niet de desktop tutorials gebruiken omdat de Jetson op een andere manier systemen gebruikt. Uiteindelijk had ik het werkend en had ik het even aan de kant gezet om met de communicatie tussen de Jetson en Xarm6 te werken. Vervolgens keerde ik een paar weken later weer terug naar ROS2 en kwam ik erachter dat het om een onbekende reden niet meer werkte. Na een lange tijd online hebben zoeken naar oplossingen vond ik geen oplossingen, dus ben ik zelf punten gaan uit testen tot het punt dat ik ook meerdere keren ROS2 opnieuw heb geïnstalleerd. Uiteindelijk kreeg ik het weer werkend maar ik had op dat moment nog geen kennis over hoe je commando's naar een Origin kan sturen. Wel wist ik hoe ik informatie uit de Origin kan ontvangen. Jammer genoeg door de gelimiteerde online kennis op het moment in het veld van ROS2 en Jetson Orin Nano heb ik niet verder kunnen uitzoeken hoe ik verder kan. Uiteindelijk bleek het dat ROS2 een bibliotheek probeerde te openen alleen kon het deze niet vinden. Mijn eerste stap was dus te achterhalen op welke locatie de bibliotheek zit. Toen kwam ik erachter dat de bibliotheek achter een gesloten document zat hiervoor was het dus niet mogelijk om de bibliotheek te verplaatsen naar de juiste plek. Om die reden moet de lijn `export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp` dit zorgt ervoor dat er door alle documenten wordt gezocht naar de juiste bibliotheek en deze wordt onder de naam gezet die ROS2 nodig heeft.

Conclusie

Over het algemeen vond ik dit een ontzettend leuk project ook was het een hele fijne werksfeer. Echter had ik wel het idee dat ik soms te veel bezig was met het helpen van andere. Dit heeft voor mijn gevoel het project veel vertraagd. Daarbij heeft het feit dat er niemand aanwezig was met de kennis van de systemen waar ik mee moest werken ook niet geholpen met de vooruitgang. Voor een groot deel had ik het idee dat ik direct in het diepe werd gegooid gezien ik onderdelen kreeg zonder een duidelijk uitleg en het zelf moest uitzoeken. Dit heeft er echter wel voor gezorgd dat ik heel veel heb geleerd van bijvoorbeeld beeldherkenning, Linux devices en ROS. Ook was het fijn om te weten hoe een onderhoud proces verloopt. Al was het wel jammer dat het onderhoud proces nodig was. Over het algemeen vond ik het een heel leerzaam project ookal had het veel obstakels. Daarbij heeft het onderhoud proces mij echter ook heel veel geholpen met mijn telefoon angst omdat veel van dit proces over de telefoon ging. Verder zal ik wel heel graag in overleg met het HVA verder willen werken aan dit project omdat ik het persoonlijk vervelend vind om een on afgemaakt project af te leveren.